A Garden Centre Database

Scenario

Bloomfield Garden Centre is a well-established local business that operates within a suburban area noted for its community engagement and focus on green living. The garden centre occupies a sizable area, providing a broad selection of plants and related products to its customers.

Aside from selling plants, Bloomfield also hosts weekly workshops and seasonal events focused on sustainable gardening practices, DIY home garden projects, and family-friendly activities.

The staff at Bloomfield Garden Centre have diverse roles ranging from customer service to inventory management. For example, staff act as sales associates and advisors who help customers select the best plants for their gardens and offer tips for plant care.

As the business has expanded, the need for an efficient management system has become apparent. The current method of record keeping is mostly paper-based and manually managed. This has led to inefficiencies and the potential for errors.

The introduction of a database system is seen as critical to managing the growing inventory and customer base efficiently. This system would handle various business operations such as tracking sales, managing employee schedules and customer engagement events. The company expects the system to improve overall business management and customer satisfaction.

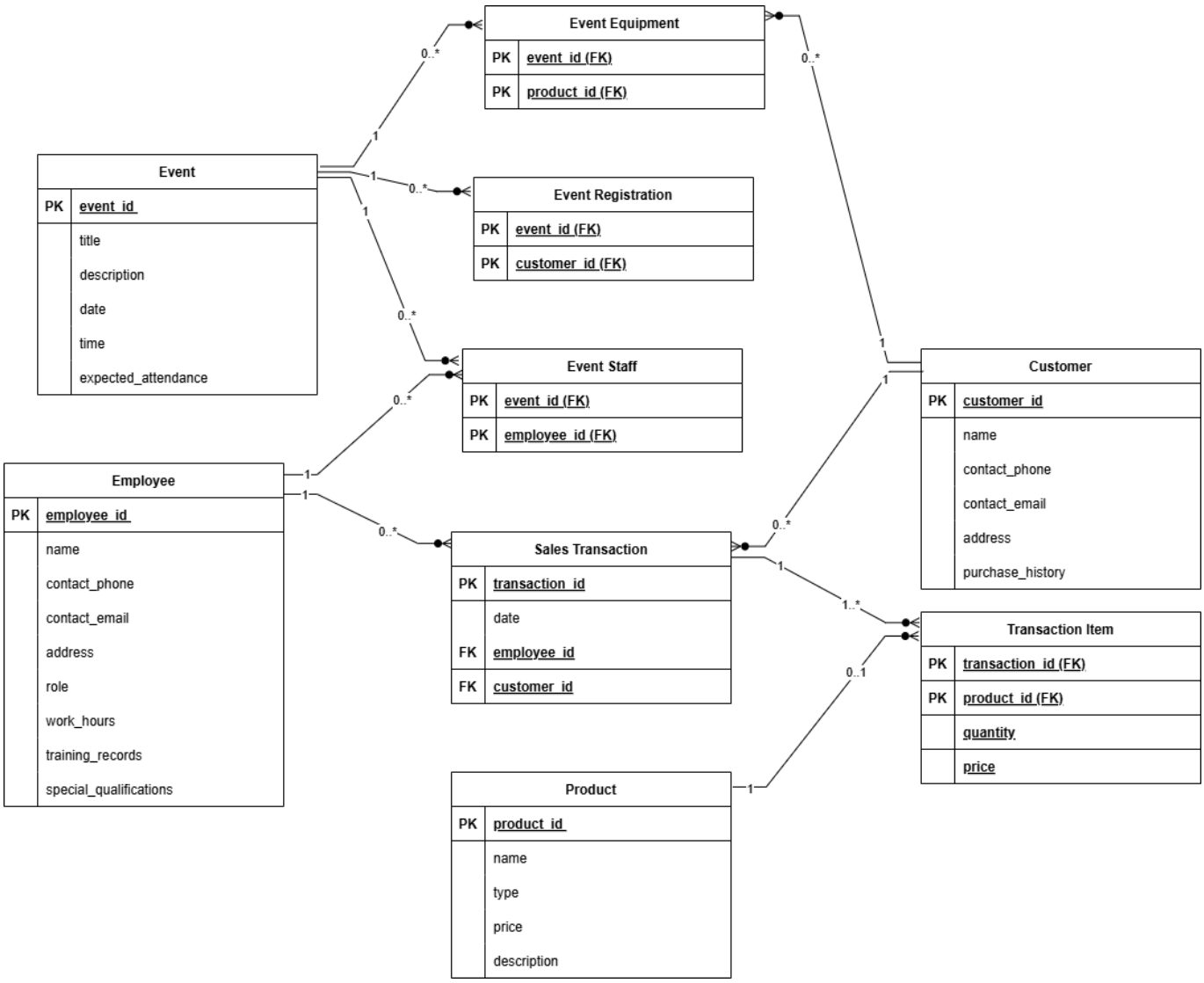
Task 1: Create an ER Diagram.

This section analyses the Database systems requirements for Bloomfield Garden center and examines the created Entity relationships Diagram.

Bloomfield garden center is a local business that runs its operations in a suburban area. They love to engage with the community and make life greener. They sell plants, gardening tools, and accessories. They also run special activities such as workshops and events about sustainable gardening.

The business is struggling to maintain efficient data because they store and keep their records using papers and everything by hands. This method has become slow and full of errors. One way to solve such a problem is to replace the old method by Creating database system to ensure faster, efficient, and more reliable record keeping.

An Entities Relationships Diagram will be created representing entities, their relationships and the cardinalities within the database system.



The Entity Relationship Diagram above is designed to help the business in managing the inventory ,employee, sales transactions, customers, and events efficiently. The diagram included nine entities. And each entity is made to address specific requirements needed by the business.

The product entity represents information about the products. Including the name, type, price, and a brief description of the product. Making it easier to record any information about a certain product.

The employee entity manages the staff data. This involves the name, contact details, address, role, work hours, training records, and special qualifications. Employee entity help in managing the employee information. The customer entity holds details about customer activities, such as name, contact details, and address. In Event entity, workshops and events are managed efficiently. Event entity records title, description, date, time, and expected attendance. Ensuring that event details are stored and can be accessed easily for scheduling purposes and future activities. The entity of sales transactions records data about sales. Such as date, purchased items, quantities, prices, and information about the employee who contributed in processing the sale. Sales transaction links between employee and customer entities by using foreign keys.

Transaction item, event staff, event equipment, and event registration are four junction entities that represent many to many relationships. Transaction item connects between sales transaction and product to track prices and quantities for each item that gets sold. Event staff connects between event and employee to find out the role of each employee in a certain event. Event equipment connects between event and product for the use of equipment. Event registration links between event and customer for the customers who participate in events. Junction entities represent complex entities which can help the business in storing the data in the database system without duplication.

Cardinalities define the relationships between entities by using UML. for example, the cardinality 1 to 0..* between customer and sales transaction means that the customer can have many transactions. The relationship between sales transaction and transaction item (1 to 1) means that each transaction could include at least one item. 1 to 0 in event and event staff means that an event could involve multiple staff members. 0..* to 1 between event registration and customer means that a customer can register in many events. The 3NF is achieved in the provided diagram, guaranteed efficiency and data integrity and each entity has a primary key.

Task 2: Data Dictionary.

Entity	Name of Attribute	Type of Data	Constraints	Field length	Key
Product	product_id	integer	Unique, not null		PK
Product	name	varchar	Not null	100	
Product	type	varchar	Not null	50	
Product	price	decimal	Not null	10,2	
Product	description	varchar	Not null	500	
Employee	employee_id	integer	Unique, not null		PK
Employee	name	varchar	Not null	100	
Employee	contact_phone	varchar	Not null	15	
Employee	contact_email	varchar	Not null	100	
Employee	address	varchar	Not null	200	
Employee	role	varchar	Not null	50	
Employee	work_hours	varchar	Not null	50	
Employee	training_records	text			
Employee	special_qualification	text			
Customer	customer_id	integer	Unique, not null		PK
Customer	name	varchar	Not null	100	
Customer	contact_phone	varchar	Not null	15	
Customer	contact_email	varchar	Not null	100	
Customer	address	varchar	Not null	200	
Customer	purchase_history	text			

Entity	Name of Attribute	Type of Data	Constraints	Field length	Key
Event	event_id	integer	Unique, not null		PK
Event	title	varchar	Not null	100	
Event	description	varchar	Not null	500	
Event	date	date	Not null		
Event	time	time	Not null		
Event	expected_attendance	integer	Not null		
S/Transaction	transaction_id	integer	Unique, not null		PK
S/Transaction	date	date	Not null		
S/Transaction	employee_id	integer	Not null		FK to employee
S/Transaction	customer_id	integer			FK to customer
Trans/Item	transaction_id	integer	Not null		Compound Key+FK to S/T
Trans/Item	product_id	integer	Not null		Compound Key+FK to product
Trans/Item	quantity	integer	Not null, default		
Trans/Item	price	decimal	Not null	10.2	
Event Staff	event_id	integer	Not null		Compound Key+FK to event
Event Staff	employee_id	integer	Not null		Compound Key+FK to employee
Event Registration	event_id	integer	Not null		Compound

					Key+FK to event
Event Registration	customer_id	integer	Not null		Compound Key+FK to customer
Event Equipment	event_id	integer	Not null		Compound Key+FK to event
Event Equipment	product_id	integer	Not null		Compound Key+FK to product

Assumptions:

1- names, such as name,title = 100 characters for long names

2- descriptions= 500 characters to provide full detailed description about a product

3- phone numbers= 15 characters to include the international phone number format.

4- email addresses= 100 characters are the standard for an email address.

5- address= 200 characters to include the full detailed addresses such as the street name or apartment number.

6- work hours + role= 50 characters are chosen at max to describe the schedule and duties for each employee.

7- for all fields that have (text) data type, text is a data type that allows for long data. For example, purchase history, transactions made, and summary.

Constraints + data types:

1- for attributes like employee id, customer id,...., integer is the chosen type of data because the id is a whole number regardless of the length. Constraint like unique means that the id number must be different and unique. While not null means that the field must not be empty. It must have a value.

2- for the price attribute, the chosen type is decimal (10,2) to represent the price that has a decimal value.

3- date and time attributes are represented by (date,time) type of data for schedules and transaction dates recording.

Task 3: Database Creation.

SQL Commands:

```
-- Product Table

CREATE TABLE IF NOT EXISTS Product (

    product_id INTEGER PRIMARY KEY,

    name VARCHAR(100) NOT NULL,

    type VARCHAR(50) NOT NULL,

    price DECIMAL(10,2) NOT NULL,

    description VARCHAR(500) NOT NULL

);



-- Employee Table

CREATE TABLE IF NOT EXISTS Employee (

    employee_id INTEGER PRIMARY KEY,

    name VARCHAR(100) NOT NULL,

    contact_phone VARCHAR(15) NOT NULL,

    contact_email VARCHAR(100) NOT NULL,
```

```
    address VARCHAR(200) NOT NULL,
    role VARCHAR(50) NOT NULL,
    work_hours VARCHAR(50) NOT NULL,
    training_records TEXT,
    special_qualifications TEXT
);


-- Customer Table
CREATE TABLE IF NOT EXISTS Customer (
    customer_id INTEGER PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    contact_phone VARCHAR(15) NOT NULL,
    contact_email VARCHAR(100) NOT NULL,
    address VARCHAR(200) NOT NULL,
    purchase_history TEXT
);
```

```
-- Event Table

CREATE TABLE IF NOT EXISTS Event (

    event_id INTEGER PRIMARY KEY,

    title VARCHAR(100) NOT NULL,

    description VARCHAR(500) NOT NULL,

    date DATE NOT NULL,

    time TIME NOT NULL,

    expected_attendance INTEGER NOT NULL

);



-- SalesTransaction Table

CREATE TABLE IF NOT EXISTS SalesTransaction (

    transaction_id INTEGER PRIMARY KEY,

    date DATE NOT NULL,

    employee_id INTEGER NOT NULL,

    customer_id INTEGER,

    FOREIGN KEY (employee_id) REFERENCES Employee(employee_id),
```

```
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);


-- TransactionItem Table
CREATE TABLE IF NOT EXISTS TransactionItem (
    transaction_id INTEGER,
    product_id INTEGER,
    quantity INTEGER NOT NULL DEFAULT 1,
    price DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (transaction_id, product_id),
    FOREIGN KEY (transaction_id) REFERENCES SalesTransaction(transaction_id),
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
);


-- EventStaff Table
CREATE TABLE IF NOT EXISTS EventStaff (
    event_id INTEGER,
```

```
    employee_id INTEGER,
    PRIMARY KEY (event_id, employee_id),
    FOREIGN KEY (event_id) REFERENCES Event(event_id),
    FOREIGN KEY (employee_id) REFERENCES Employee(employee_id)
);


-- EventEquipment Table
CREATE TABLE IF NOT EXISTS EventEquipment (
    event_id INTEGER,
    product_id INTEGER,
    PRIMARY KEY (event_id, product_id),
    FOREIGN KEY (event_id) REFERENCES Event(event_id),
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
);


-- EventRegistration Table
CREATE TABLE IF NOT EXISTS EventRegistration (
```

```
    event_id INTEGER,
    customer_id INTEGER,
    PRIMARY KEY (event_id, customer_id),
    FOREIGN KEY (event_id) REFERENCES Event(event_id),
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);
-- Product Data
INSERT INTO Product (product_id, name, type, price, description)
VALUES (1, 'Rose Plant', 'Plant', 15.99, 'Red rose bush, hardy perennial'),
       (2, 'Garden Shovel', 'Tool', 25.50, 'Durable steel shovel'),
       (3, 'Fertilizer', 'Supply', 10.75, 'Organic fertilizer for plants'),
       (4, 'Pruning Shears', 'Tool', 18.99, 'Sharp pruning tool'),
       (5, 'Lawn Mower', 'Equipment', 199.99, 'Electric lawn mower');


-- Employee Data
INSERT INTO Employee (employee_id, name, contact_phone, contact_email, address,
role, work_hours, training_records, special_qualifications)
```

VALUES (1, 'John Doe', '123-456-7890', 'john@example.com', '123 Green St', 'Gardener', 'Mon-Fri, 9-5', 'Plant Care 2025', 'Certified Horticulturist'),

(2, 'Jane Smith', '234-567-8901', 'jane@example.com', '456 Oak Ave', 'Sales Associate', 'Tue-Sat, 10-6', 'Sales Training 2025', NULL),

(3, 'Mike Brown', '345-678-9012', 'mike@example.com', '789 Pine Rd', 'Manager', 'Mon-Fri, 8-4', 'Management 2025', 'Pesticide License'),

(4, 'Sarah Lee', '456-789-0123', 'sarah@example.com', '101 Maple Ln', 'Cashier', 'Mon-Sat, 9-5', NULL, NULL),

(5, 'Tom Wilson', '567-890-1234', 'tom@example.com', '202 Cedar St', 'Trainer', 'Mon-Wed, 10-3', 'First Aid 2025', 'Training Cert');

-- Customer Data

INSERT INTO Customer (customer_id, name, contact_phone, contact_email, address, purchase_history)

VALUES (1, 'Alice Johnson', '111-222-3333', 'alice@j.com', '301 Elm St', 'Rose Plant 2025-05-21'),

(2, 'Bob Taylor', '222-333-4444', 'bob@t.com', '404 Birch Rd', 'Shovel 2025-05-21'),

(3, 'Clara White', '333-444-5555', 'clara@w.com', '505 Spruce Ave', 'Fertilizer 2025-05-21'),

(4, 'David Black', '444-555-6666', 'david@b.com', '606 Oak Ln', NULL),

(5, 'Emma Green', '555-666-7777', 'emma@g.com', '707 Pine St', 'Pruning Shears 2025-05-21');

-- Event Data

INSERT INTO Event (event_id, title, description, date, time, expected_attendance)

VALUES (1, 'Spring Planting', 'Learn to plant seasonal flowers', '2025-05-22', '14:00:00', 50),

(2, 'Gardening Basics', 'Introduction to gardening techniques', '2025-05-25', '10:00:00', 30),

(3, 'Tool Maintenance', 'How to maintain garden tools', '2025-05-28', '13:00:00', 40),

(4, 'Organic Farming', 'Tips for organic gardening', '2025-05-30', '11:00:00', 60),

(5, 'Tree Care', 'Pruning and caring for trees', '2025-06-01', '15:00:00', 45);

-- SalesTransaction Data

INSERT INTO SalesTransaction (transaction_id, date, employee_id, customer_id)

VALUES (1, '2025-05-21', 1, 1),

```
    (2, '2025-05-21', 2, 2),
    (3, '2025-05-21', 3, 3),
    (4, '2025-05-21', 4, 4),
    (5, '2025-05-21', 5, 5);

-- TransactionItem Data
INSERT INTO TransactionItem (transaction_id, product_id, quantity, price)
VALUES (1, 1, 2, 15.99),
    (2, 2, 1, 25.50),
    (3, 3, 3, 10.75),
    (4, 4, 1, 18.99),
    (5, 5, 1, 199.99);

-- EventStaff Data
INSERT INTO EventStaff (event_id, employee_id)
VALUES (1, 1),
    (2, 2),
```

```
    (3, 3),
    (4, 4),
    (5, 5);


-- EventEquipment Data
INSERT INTO EventEquipment (event_id, product_id)
VALUES (1, 2),
    (2, 4),
    (3, 2),
    (4, 3),
    (5, 5);


-- EventRegistration Data
INSERT INTO EventRegistration (event_id, customer_id)
VALUES (1, 1),
    (2, 2),
```

(3, 3),

(4, 4),

(5, 5);

Sample Data:

Database Structure | Browse Data | Edit Pragmas | Execute SQL

Table: Product

Filter in any column

	product_id	name	type	price	description
	Filter	Filter	Filter	Filter	Filter
1	1	Rose Plant	Plant	15.99	Red rose bush, hardy perennial
2	2	Garden Shovel	Tool	25.5	Durable steel shovel
3	3	Fertilizer	Supply	10.75	Organic fertilizer for plants
4	4	Pruning Shears	Tool	18.99	Sharp pruning tool
5	5	Lawn Mower	Equipment	199.99	Electric lawn mower

Database Structure | Browse Data | Edit Pragmas | Execute SQL

Table: Customer

Filter in any column

	customer_id	name	contact_phone	contact_email	address	purchase_history
	Filter	Filter	Filter	Filter	Filter	Filter
1	1	Alice Johnson	111-222-3333	alice@j.com	301 Elm St	Rose Plant 2025-05-21
2	2	Bob Taylor	222-333-4444	bob@t.com	404 Birch Rd	Shovel 2025-05-21
3	3	Clara White	333-444-5555	clara@w.com	505 Spruce Ave	Fertilizer 2025-05-21
4	4	David Black	444-555-6666	david@b.com	606 Oak Ln	NULL
5	5	Emma Green	555-666-7777	emma@g.com	707 Pine St	Pruning Shears 2025-05-21

Database Structure | Browse Data | Edit Pragmas | Execute SQL

Table: Employee

Filter in any column

	employee_id	name	contact_phone	contact_email	address	role	work_hours	training_records	special_qualifications
1	1	John Doe	123-456-7890	john@example.com	123 Green St	Gardener	Mon-Fri, 9-5	Plant Care 2025	Certified Horticulturi
2	2	Jane Smith	234-567-8901	jane@example.com	456 Oak Ave	Sales Associate	Tue-Sat, 10-6	Sales Training 2025	NULL
3	3	Mike Brown	345-678-9012	mike@example.com	789 Pine Rd	Manager	Mon-Fri, 8-4	Management 2025	Pesticide License
4	4	Sarah Lee	456-789-0123	sarah@example.com	101 Maple Ln	Cashier	Mon-Sat, 9-5	NULL	NULL
5	5	Tom Wilson	567-890-1234	tom@example.com	202 Cedar St	Trainer	Mon-Wed, 10-3	First Aid 2025	Training Cert

New Database | Open Database | Write Changes | Revert Changes | Undo | Open Project | Save Project | Attach Datab

Database Structure | Browse Data | Edit Pragmas | Execute SQL

Table: Event

Filter in any column

	event_id	title	description	date	time	expected_attendance
1	1	Spring Planting	Learn to plant seasonal flowers	2025-05-22	14:00:00	50
2	2	Gardening Basics	Introduction to gardening techniques	2025-05-25	10:00:00	30
3	3	Tool Maintenance	How to maintain garden tools	2025-05-28	13:00:00	40
4	4	Organic Farming	Tips for organic gardening	2025-05-30	11:00:00	60
5	5	Tree Care	Pruning and caring for trees	2025-06-01	15:00:00	45

Database Structure | Browse Data

Table: EventEquipment

	event_id	product_id
	Filter	Filter
1	1	2
2	2	4
3	3	2
4	4	3
5	5	5

Database Structure | Browse Data

Table: EventRegistration

event_id	customer_id
Filter	Filter
1	1
2	2
3	3
4	4
5	5

Table: EventStaff

	event_id	employee_id
	Filter	Filter
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

Table: SalesTransaction

	transaction_id	date	employee_id	customer_id
	Filter	Filter	Filter	Filter
1	1	2025-05-21	1	1
2	2	2025-05-21	2	2
3	3	2025-05-21	3	3
4	4	2025-05-21	4	4
5	5	2025-05-21	5	5

Database Structure | Browse Data | Edit Pragmas | Execute

Table: TransactionItem

	transaction_id	product_id	quantity	price
	Filter	Filter	Filter	Filter
1	1	1	2	15.99
2	2	2	1	25.5
3	3	3	3	10.75
4	4	4	1	18.99
5	5	5	1	199.99

Task 4: Rule Based Access Control.

The selected roles:

Sales: this role is assigned to the sales staff who are responsible for handling customer inquiries and transactions. The staff needs to access the database but they should not modify the inventory or delete transactions.

Admin: this role is assigned to the managers who are responsible for tracking operations. They can access the database in order to write reports and make corrections.

The selected users:

sales_user: for sales role

admin_user: for admin role.

NOTE: The tool used for creating commands is SQLite and it does not support rule based access control and commands like (CREATE ROLE-CREATE USER) that are used to create them.

Therefore, admin and user roles will be defined by concepts. In the following example, an sql will be used to create them.

The selected tables for access control are:

1-Product: this table includes inventory data

2-salesTransaction: this table includes sales data

Privilege selected for sales role are:

1- SELECT. Because product details must be accessed by the sales staff in order to serve the customer. However, the sales staff must not modify the inventory to avoid major changes to the stock.

2- SELECT, INSERT. Because after each transaction made by customers, the sales staff must record the transaction and view the transaction that was made in a certain day to verify it. However, they are still required to not modify the transaction or delete the transaction to avoid errors and potential fraud.

Privilege selected for admin role are:

1- SELECT, INSERT, UPDATE, DELETE.

1- Because adding new products to the inventory requires full control by the admins.

2- changing prices requires the admins

3- removing items that are not being sold or not exist any more requires the admins

4- Viewing details about products requires the admins to generate reports and evaluate the current situation within the inventory.

Access control:

For sales role:

```
-- View for Product table (SELECT only)

 CREATE VIEW sales_product_view AS

 SELECT product_id, name, type, price, description

 FROM Product;

-- View for SalesTransaction table (SELECT only, for viewing)

CREATE VIEW sales_transaction_view AS

SELECT transaction_id, date, employee_id, customer_id

FROM SalesTransaction

WHERE date = '2025-05-21';
```

NOTE: Admin role can access both tables.

The following sql will show how commands can be run in MySQL (that supports Rule based access control).

```
GRANT SELECT ON bloomfield_garden.Product TO sales;

GRANT SELECT, INSERT ON bloomfield_garden.SalesTransaction TO sales;

GRANT SELECT, INSERT, UPDATE, DELETE ON bloomfield_garden.Product TO admin;

GRANT SELECT, INSERT, UPDATE, DELETE ON bloomfield_garden.SalesTransaction TO admin;

FLUSH PRIVILEGES;
```

Outputs:

for sales user :

SQL 1*

```
164   VIEW sales_product_view AS
165   product_id, name, type, price,
166   oduct;
167
168    for SalesTransaction table (SE
169   VIEW sales_transaction_view AS
170   transaction_id, date, employee_
171   lesTransaction
172   ate = '2025-05-21';
173
174   CT on Product (via view)
175   * FROM sales_product_view;
```

	product_id	name	type	
1	1	Rose Plant	Plant	15
2	2	Garden Shovel	Tool	25
3	3	Fertilizer	Supply	10
4	4	Pruning Shears	Tool	18
5	5	Lawn Mower	Equipment	19

```
Execution finished without errors.
Result: 5 rows returned in 10ms
At line 174:
-- SELECT on Product (via view)
SELECT * FROM sales_product_view;
```

Sales user can see the details of the product from the picture provided above.

re Browse Data Edit Pragmas Execute SQL

SQL 1*

```
163   View for Product table (SELECT
164 VIEW sales_product_view AS
165 product_id, name, type, price,
166 oduct;
167
168  for SalesTransaction table (SE
169 VIEW sales_transaction_view AS
170 transaction_id, date, employee_
171 lesTransaction
172 ate = '2025-05-21';
173
174 * FROM sales_transaction_view;
```

	transaction_id	date	employee_id	cust
1	1	2025-05-21	1	1
2	2	2025-05-21	2	2
3	3	2025-05-21	3	3
4	4	2025-05-21	4	4
5	5	2025-05-21	5	5

```
Execution finished without errors.
Result: 5 rows returned in 7ms
At line 174:
SELECT * FROM sales_transaction_view;
```

The sales user can see the transaction according to the picture provided above.

ure | Browse Data | Edit Pragmas | Execute SQL

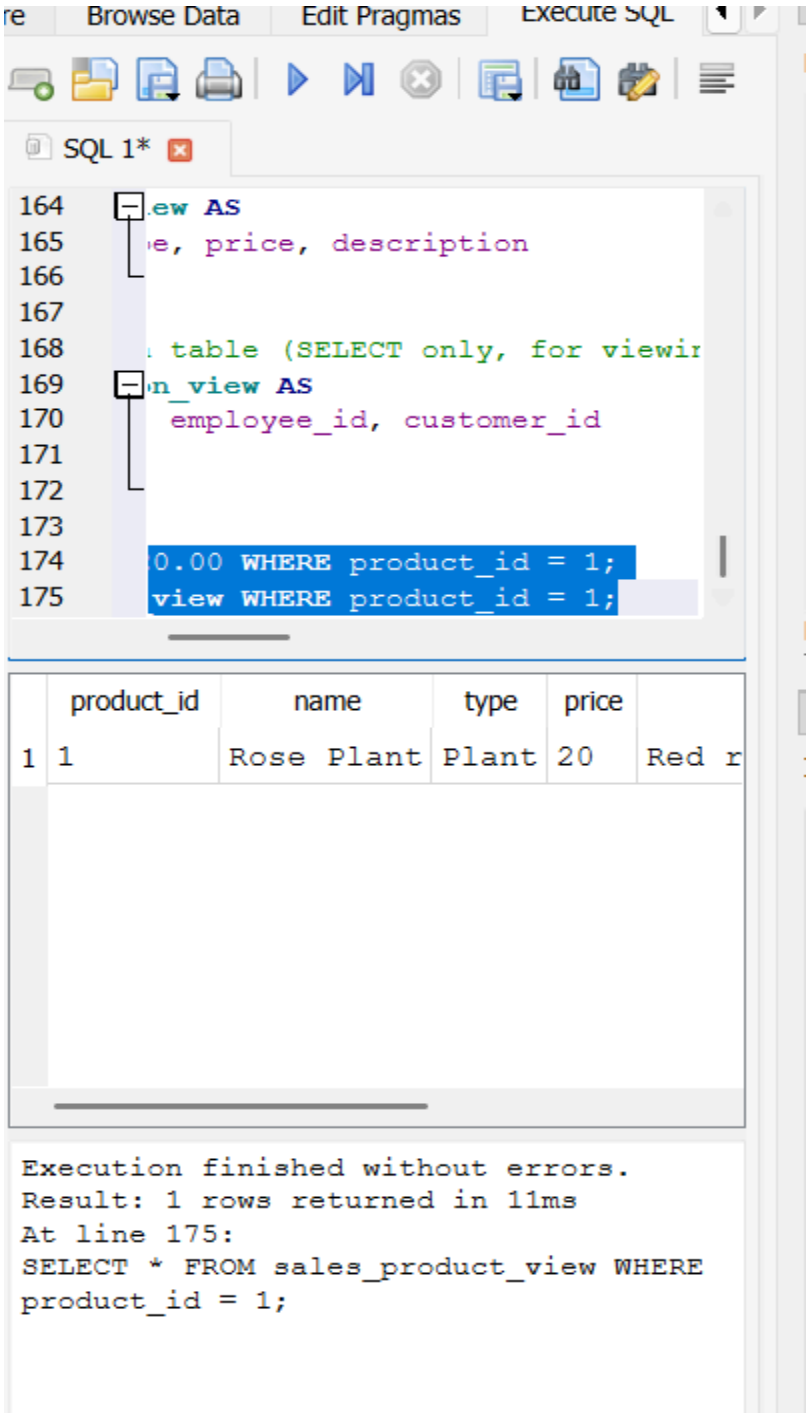
SQL 1*

```
165   product_id, name, type, price,
166   oduct;
167
168   for SalesTransaction table (SE
169   VIEW sales_transaction_view AS
170   transaction_id, date, employee_
171   lesTransaction
172   ate = '2025-05-21';
173
174   INTO SalesTransaction (transact
175   (6, '2025-05-21', 1, 1);
176   * FROM sales_transaction_view;
```

	transaction_id	date	employee_id	cust
1	1	2025-05-21	1	1
2	2	2025-05-21	2	2
3	3	2025-05-21	3	3
4	4	2025-05-21	4	4
5	5	2025-05-21	5	5
6	6	2025-05-21	1	1

```
Execution finished without errors.
Result: 6 rows returned in 17ms
At line 176:
SELECT * FROM sales_transaction_view;
```

Sales user can insert the last transaction made.



Because sqlite does not support RBAC, the update will not be shown but it will succeed in MySQL.

For admin user:

re | Browse Data | Edit Pragmas | Execute SQL

SQL 1*

```
164 CREATE VIEW sales_product_view
165 SELECT product_id, name, type,
166 FROM Product;
167
168 -- View for SalesTransaction ta
169 CREATE VIEW sales_transaction_v
170 SELECT transaction_id, date, em
171 FROM SalesTransaction
172 WHERE date = '2025-05-21';
173
174 -- SELECT on Product
175 SELECT * FROM Product;
```

	product_id	name	type	
1	1	Rose Plant	Plant	20
2	2	Garden Shovel	Tool	25
3	3	Fertilizer	Supply	10
4	4	Pruning Shears	Tool	18
5	5	Lawn Mower	Equipment	19

```
Execution finished without errors.
Result: 5 rows returned in 17ms
At line 174:
-- SELECT on Product
SELECT * FROM Product;
```

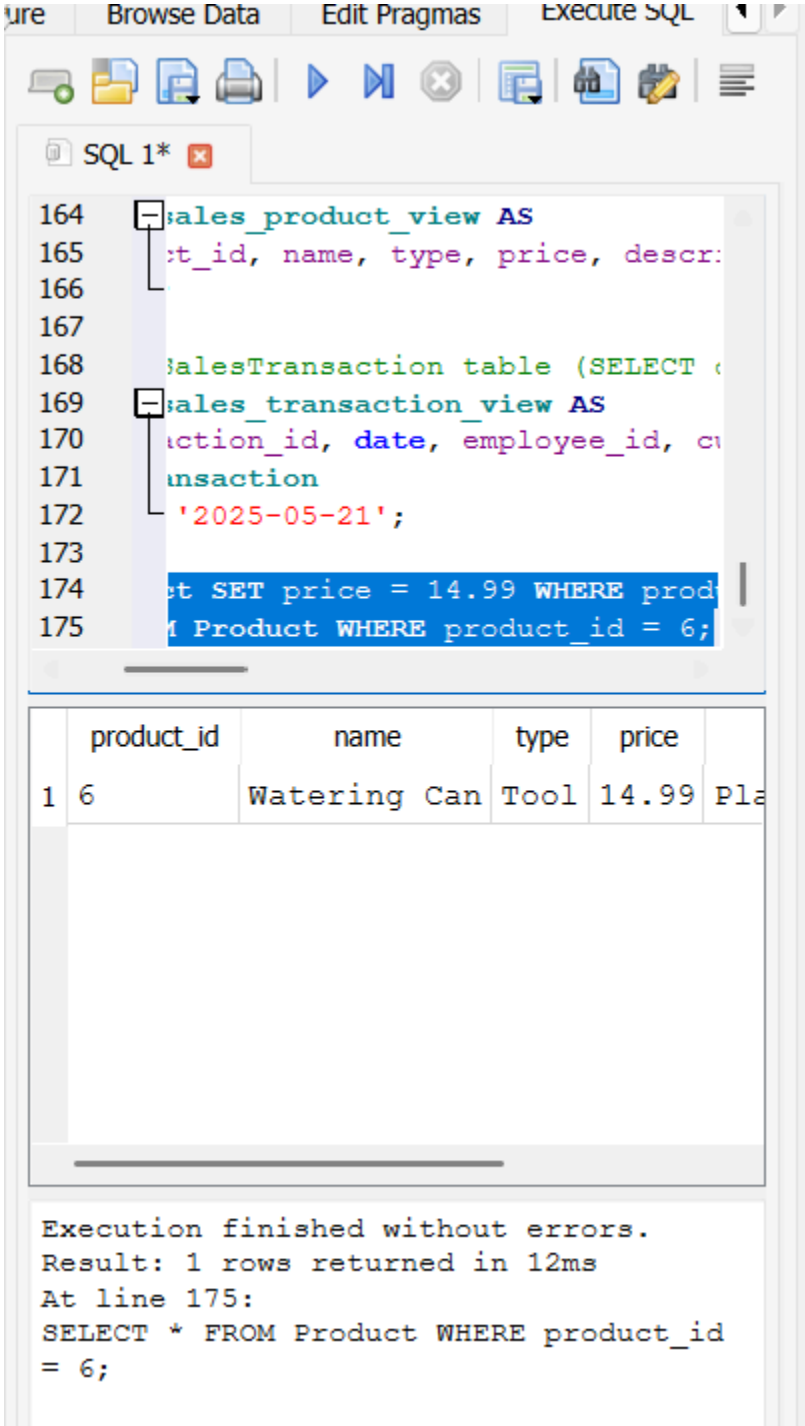
Admin can see the details of the product.

```
165   ame, type, price, description
166
167
168   nsaction table (SELECT only, f
169   ansaction_view AS
170   d, date, employee_id, customer_
171   n
172   5-21';
173
174   (product_id, name, type, price
175   Can', 'Tool', 12.99, 'Plastic
176   t WHERE product_id = 6;
```

	product_id	name	type	price	
1	6	Watering Can	Tool	12.99	Pla

```
Execution finished without errors.
Result: 1 rows returned in 11ms
At line 176:
SELECT * FROM Product WHERE product_id
= 6;
```

Admin can add a product.



Admin can modify the product.

```
166
167
168   SalesTransaction table (SELECT
169   Sales_transaction_view AS
170   action_id, date, employee_id, cu
171   ansaction
172   '2025-05-21';
173
174   Product WHERE product_id = 6;
175   M Product WHERE product_id = 6;
```

product_id	name	type	price	description

```
Execution finished without errors.
Result: 0 rows returned in 9ms
At line 175:
SELECT * FROM Product WHERE product_id
= 6;
```

Admin can delete product

```
166
167
168   on table (SELECT only, for vie
169   tion_view AS
170   e, employee_id, customer_id
171
172
173
174   T customer_id = 2 WHERE transa
175   tion WHERE transaction_id = 6;
```

	on_id	date	employee_id	customer_id
1		2025-05-21	1	2

```
Execution finished without errors.
Result: 1 rows returned in 10ms
At line 175:
SELECT * FROM SalesTransaction WHERE
transaction_id = 6;
```

Admin can modify and see transactions

Task 5: SQL Queries.

SQL 1*

```sql
-- View for SalesTransaction ta
CREATE VIEW sales_transaction_v
SELECT transaction_id, date, em
FROM SalesTransaction
WHERE date = '2025-05-21';

UPDATE Product
SET price = price * 1.10
WHERE type = 'Tool';
SELECT product_id, name, type,
FROM Product
WHERE type = 'Tool';
```

	product_id	name	type	price
1	2	Garden Shovel	Tool	28.05
2	4	Pruning Shears	Tool	20.889

```
Execution finished without errors.
Result: 2 rows returned in 12ms
At line 177:
SELECT product_id, name, type, price,
description
FROM Product
WHERE type = 'Tool';
```

this query shows the increases on all gardening tools by 10%

ure | Browse Data | Edit Pragmas | Execute SQL

SQL 1*

```sql
-- View for SalesTransaction ta
CREATE VIEW sales_transaction_v
SELECT transaction_id, date, em
FROM SalesTransaction
WHERE date = '2025-05-21';

SELECT name, type, price, descr
FROM Product
WHERE type IN ('Plant', 'Tool',
AND price = (SELECT MAX(price)
ORDER BY type;
```

	name	type	price	de
1	Lawn Mower	Equipment	199.99	Electric

```
Execution finished without errors.
Result: 1 rows returned in 9ms
At line 174:
SELECT name, type, price, description
FROM Product
WHERE type IN ('Plant', 'Tool',
'Supply', 'Equipment') -- Assuming
'Supply' as accessories
AND price = (SELECT MAX(price) FROM
Product WHERE type = Product.type)
ORDER BY type;
```

this query shows the most expensive product

SQL 1*

```sql
-- View for SalesTransaction ta
CREATE VIEW sales_transaction_v
SELECT transaction_id, date, em
FROM SalesTransaction
WHERE date = '2025-05-21';

SELECT p.type, SUM(ti.quantity)
FROM SalesTransaction st
JOIN TransactionItem ti ON st.t
JOIN Product p ON ti.product_id
WHERE strftime('%Y-%m', st.date
GROUP BY p.type;
```

	type	total_quantity	total_revenue
1	Equipment	1	199.99
2	Plant	2	31.98
3	Supply	3	32.25
4	Tool	2	44.49

```
Execution finished without errors.
Result: 4 rows returned in 8ms
At line 174:
SELECT p.type, SUM(ti.quantity) AS
total_quantity, SUM(ti.quantity *
ti.price) AS total_revenue
FROM SalesTransaction st
JOIN TransactionItem ti ON
st.transaction_id = ti.transaction_id
JOIN Product p ON ti.product_id =
p.product_id
WHERE strftime('%Y-%m', st.date) =
'2025-05' -- Past month: April 2025
GROUP BY p.type;
```

This query provides summaries about each month in terms of sales.

SQL 1*

```sql
-- View for SalesTransaction ta
CREATE VIEW sales_transaction_v
SELECT transaction_id, date, em
FROM SalesTransaction
WHERE date = '2025-05-21';

SELECT e.title, e.date, e.time,
FROM Event e
LEFT JOIN EventRegistration er
WHERE e.date BETWEEN '2025-05-2
GROUP BY e.event_id, e.title, e
ORDER BY e.date;
```

	title	date	time
1	Spring Planting	2025-05-22	14:00:00
2	Gardening Basics	2025-05-25	10:00:00
3	Tool Maintenance	2025-05-28	13:00:00
4	Organic Farming	2025-05-30	11:00:00
5	Tree Care	2025-06-01	15:00:00

```
Execution finished without errors.
Result: 5 rows returned in 8ms
At line 174:
SELECT e.title, e.date, e.time,
COUNT(er.customer_id) AS
registered_participants
FROM Event e
LEFT JOIN EventRegistration er ON
e.event_id = er.event_id
WHERE e.date BETWEEN '2025-05-21' AND
'2025-06-20'
GROUP BY e.event_id, e.title, e.date,
e.time
ORDER BY e.date;
```

this query shows the upcoming events

SQL 1*

```
170 SELECT transaction_id, date, em
171 FROM SalesTransaction
172 WHERE date = '2025-05-21';
173
174 SELECT e.name, SUM(ti.quantity
175        RANK() OVER (ORDER BY SU
176 FROM Employee e
177 JOIN SalesTransaction st ON e.e
178 JOIN TransactionItem ti ON st.t
179 WHERE st.date <= '2025-05-21'
180 GROUP BY e.employee_id, e.name
181 ORDER BY total_revenue DESC;
```

	name	total_revenue	revenue_rank
1	Tom Wilson	199.99	1
2	Mike Brown	32.25	2
3	John Doe	31.98	3
4	Jane Smith	25.5	4
5	Sarah Lee	18.99	5

```
Execution finished without errors.
Result: 5 rows returned in 15ms
At line 174:
SELECT e.name, SUM(ti.quantity *
ti.price) AS total_revenue,
       RANK() OVER (ORDER BY
SUM(ti.quantity * ti.price) DESC) AS
revenue_rank
FROM Employee e
JOIN SalesTransaction st ON
e.employee_id = st.employee_id
JOIN TransactionItem ti ON
st.transaction_id = ti.transaction_id
WHERE st.date <= '2025-05-21'
```

This query shows the total revenue made by each employee to find out the top performers.

References:

Atlan (2025) What is a data dictionary? Available at: https://atlan.com/what-is-a-data-dictionary/

British Standards Institution (2006) BS 7666:2006 Spatial datasets for geographical referencing. London: BSI.

Coehoorn, J. (2008) Answer to 'List of standard lengths for database fields'. Stack Overflow, 21 August. Available at: https://stackoverflow.com/a/20968

OSF Support (2023) How to make a data dictionary. Available at: https://help.osf.io/article/217-how-to-make-a-data-dictionary

SS64 (n.d.) Database field sizes. Available at: https://ss64.com/sql/syntax-field-sizes.html

(PDF) role-based access control. Available at: https://www.researchgate.net/publication/2792237_Role-Based_Access_Control

sqlite always validates its inputs Defense Against The Dark Arts. Available at:

https://www.sqlite.org/security.html

introduction to window functions Window Functions. Available at:

https://www.sqlite.org/windowfunctions.html